ROBÔS SEGUIDORES DE LINHA AUTÔNOMOS

UTFPR
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CÂMPUS PATO BRANCO

ARDUINO™

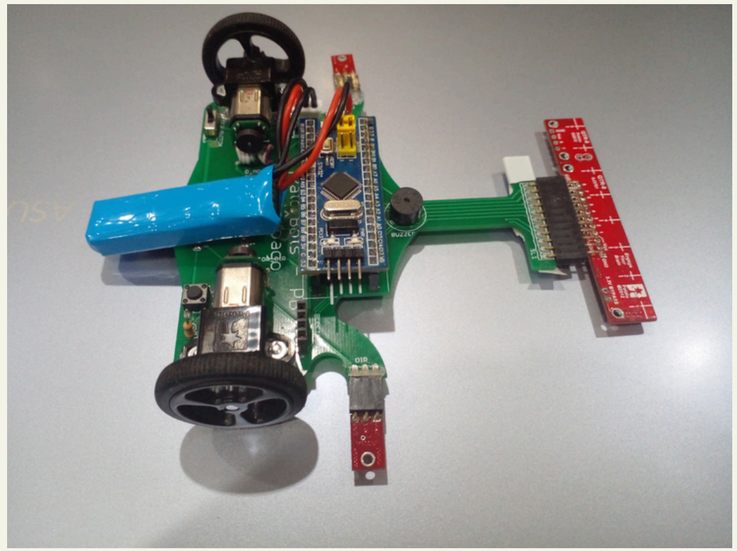
Patobots - UTFPR

GITHUB

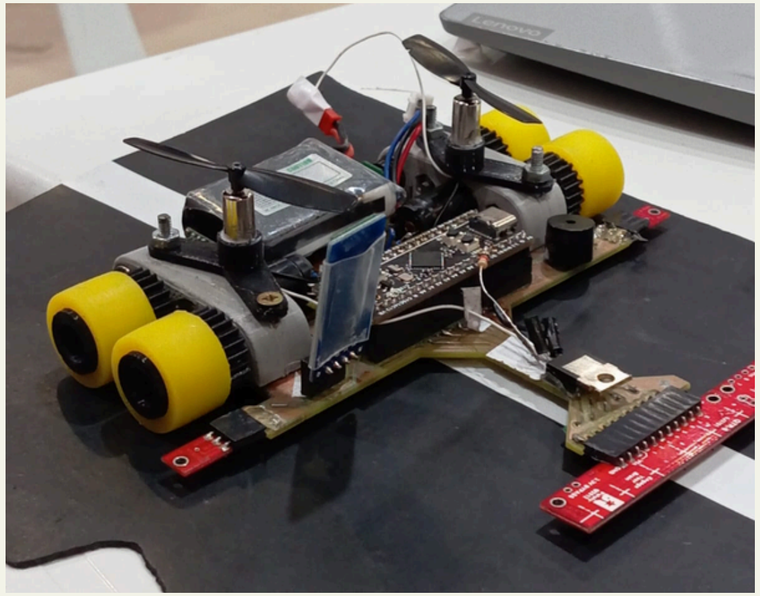


https://github.com/AmandaZippin/Curso-Seguidor-de-Linha/tree/main/Aula%201

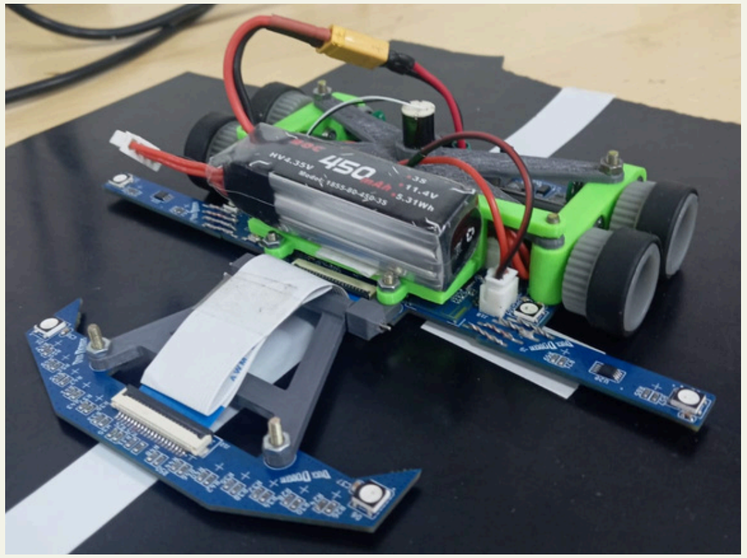
SEGUIDOR DE LINHA



Básico



com **ventoinha**



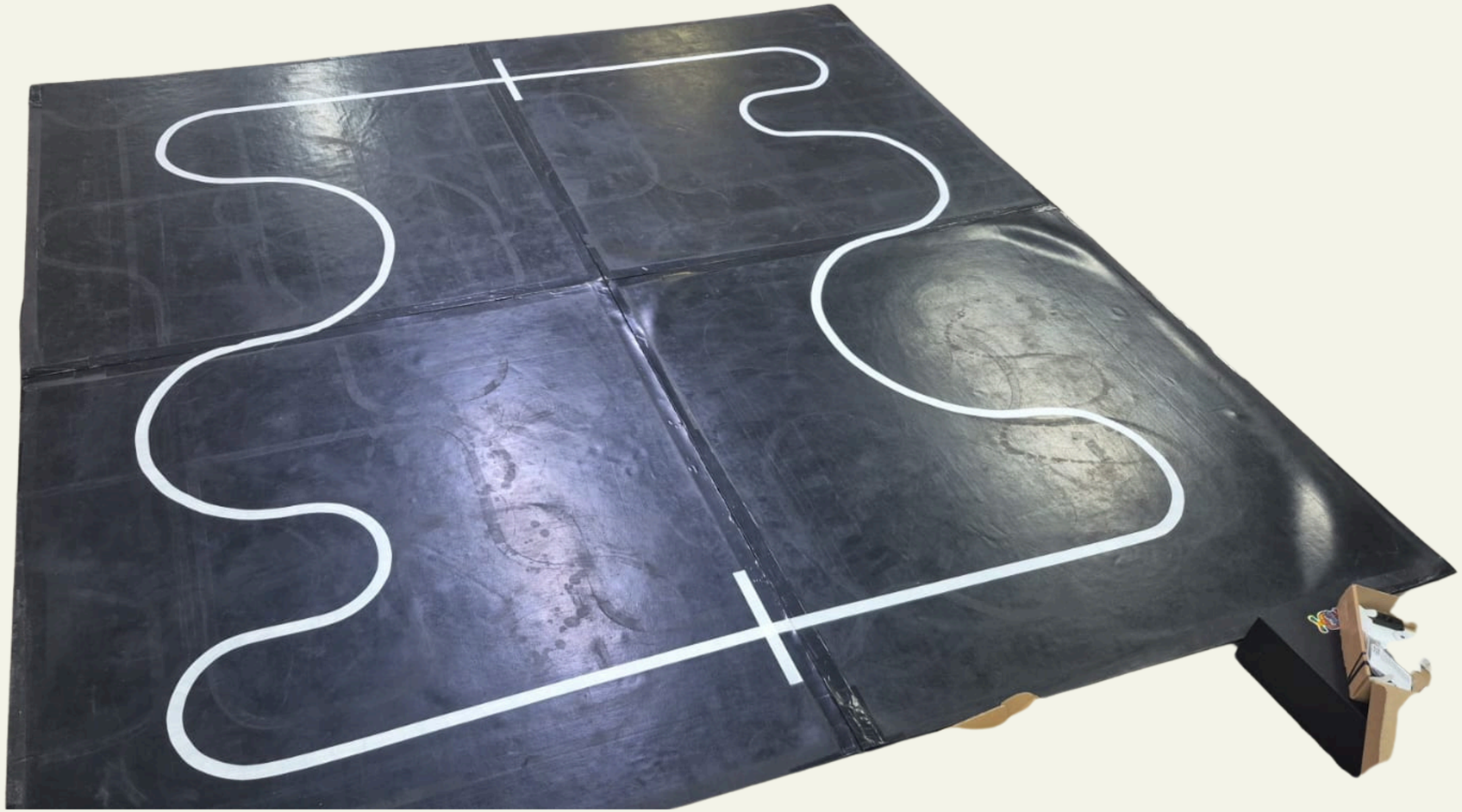
com **sucção**



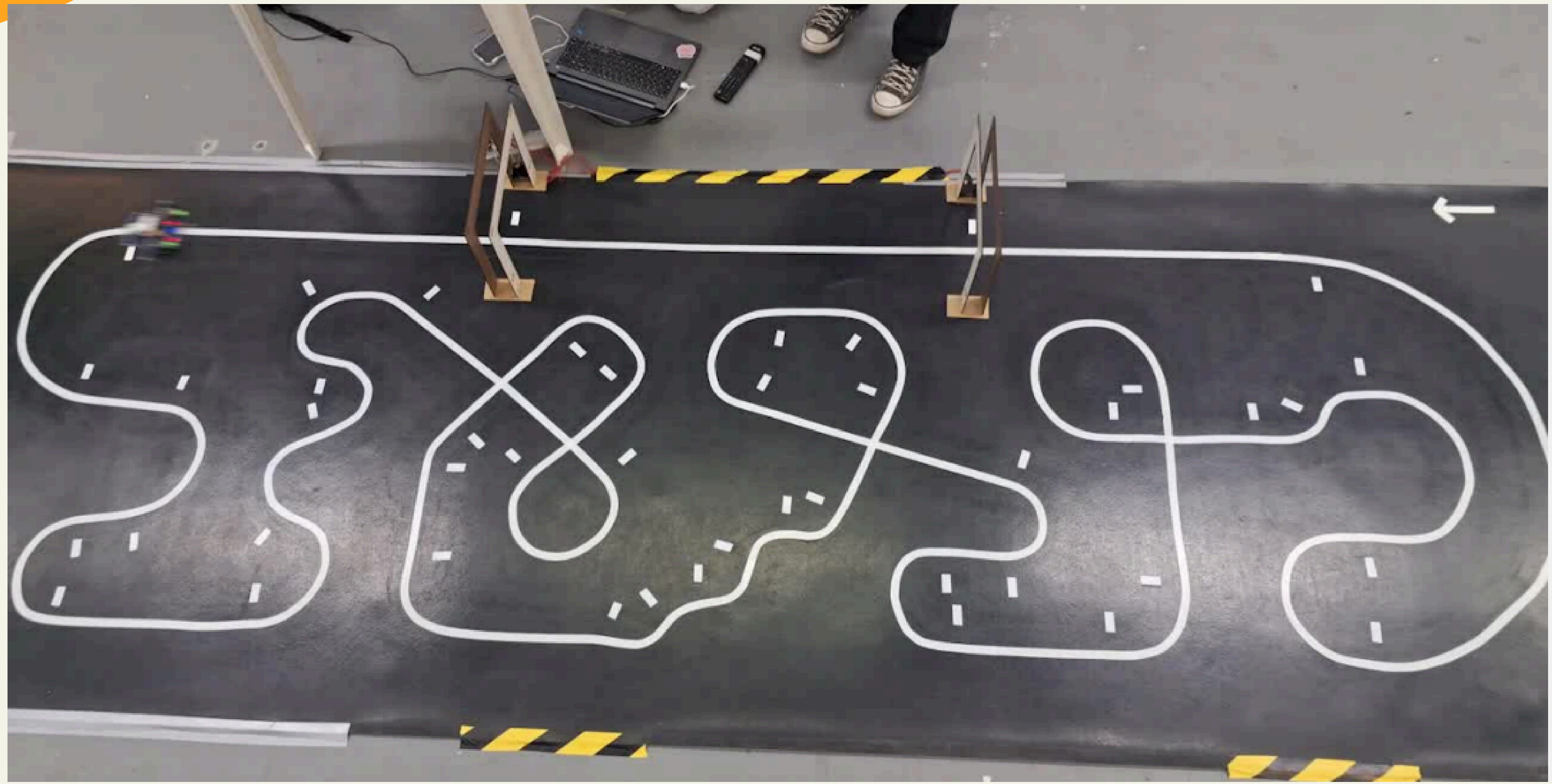
Pista do Seguidor de Linha PRO - RoboChallenge 2025



Pista do Seguidor de Linha PRO - Copa Pinhão 2026



Pista Perseguidor de Linha PRO - Copa Pinhão 2026



Pista Competitiva Seguidor de Linha PRO

FRONTAL0
QTR-8A
SG1
F/OMBRO
JP2
PE
PD
JP1
8,54
ESQ0
QTR-1A
7.4V
GND
BATERIAO
R1
10k
BLUETOOH
HC_05
DIR0
PatoBots
UTFPR-PB
Projeto de Extensão
Robótica Móvel
J3 MOTOR_E
JM-E0
MOT_E
TB6612FNG0
ROB-14450
JM-D0
MOTOR_D J2
16,15
10,71
S1
BOTAO1
BOTO
IN
OUT

Lógica do Seguidor de Linha

- **O robô detecta uma faixa no chão e ajusta sua trajetória para permanecer sobre ela;**

- **Seus sensores identificam a posição da linha e enviam sinais para o Sistema de Controle;**

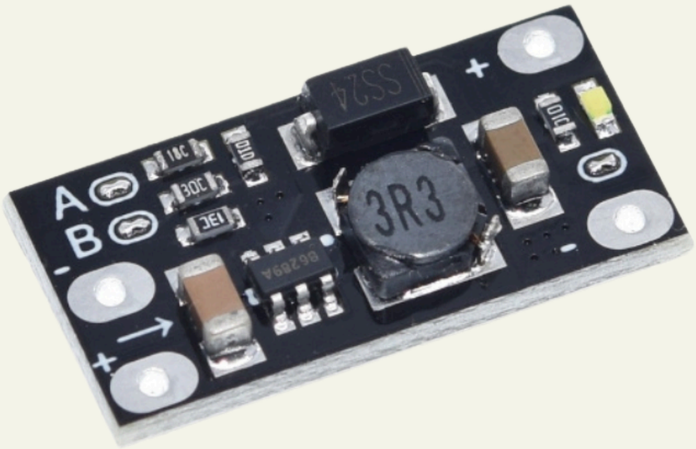
- **O Sistema de Controle ajusta os motores com base nos dados dos sensores, corrigindo sua direção.**

Cronograma

- **Step-Up**
- **Tipos de Sensores: Sensor Óptico Reflexivo e Sensor QTR**
- **Arduino NANO**
- **Motores**
- **Ponte H**

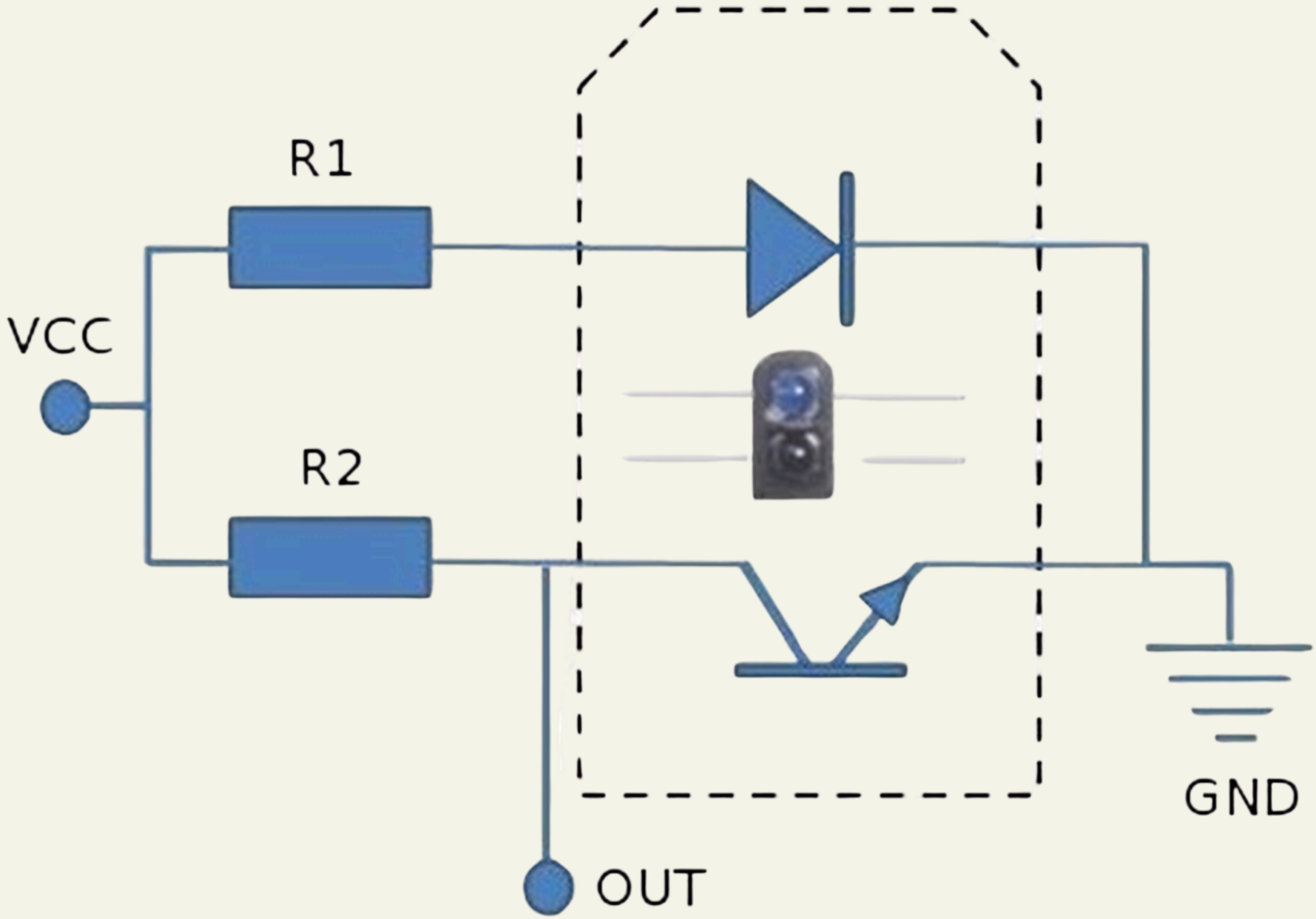
REGULADOR DE TENSÃO STEP-UP (BOOST)





STEP-UP (BOOST)

- **Têm como base semicondutores;**
- **Sua função é regular a tensão no fim do circuito elétrico (aumentar ou diminuir);**
- **Benefícios de seu uso: versátil, compacto, econômico;**
- **<u>OBS: não é capaz de GERAR energia.</u>**


R1
VCC
R2
OUT
GND

SENSOR ÓPTICO REFLEXIVO

- **O sensor é divido em dois componentes: um sensor infravermelho, e um fototransistor;**

- **Quando um objeto se aproxima do sensor, a luz infravermelha é refletida no objeto, passa para o outro lado e ativa o resistor, fazendo a leitura;**

- **Quanto mais reflexivo o material do objeto (branco), o sinal será maior, enquanto em cores de menor reflexão (preto), o sinal será próximo a zero.**

SENSOR ÓPTICO REFLEXIVO

MODOS DE LEITURA DIGITAL E ANALÓGICO

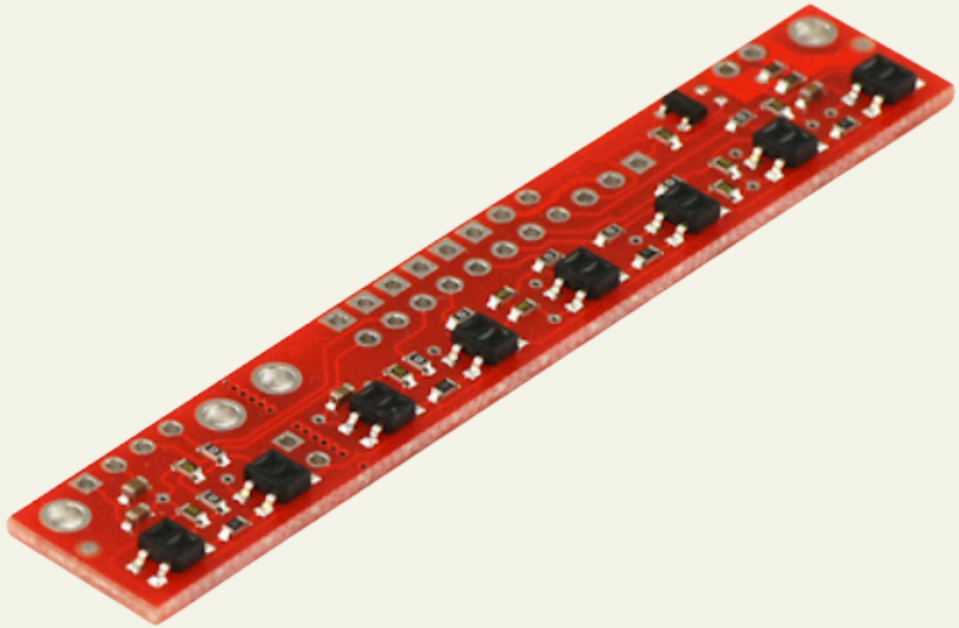
- **Modo Digital: envia o sinal de estar lendo a reflexão ou não.**
- **Modo Analógico: envia o sinal de quanto está sendo refletido pelo sensor.**

SENSOR ÓPTICO REFLEXIVO

- **Este sensor está soldado em uma placa que faz aumentar a distância a qual o sensor consegue refletir seu infravermelho, possibilitando também uma melhor precisão;**

- **Em cima da placa, tem um potenciômetro que ajusta a precisão da leitura do sensor, regulando o quanto de sinal ele envia pela distância.**

SENSOR DE REFLETÂNCIA QTR



QTR-8 A/D

- Contém oito pares de emissores e receptores infravermelhos;
- Retorna o sinal do sensor que que detectou a reflexão.

SENSOR DE REFLETÂNCIA QTR

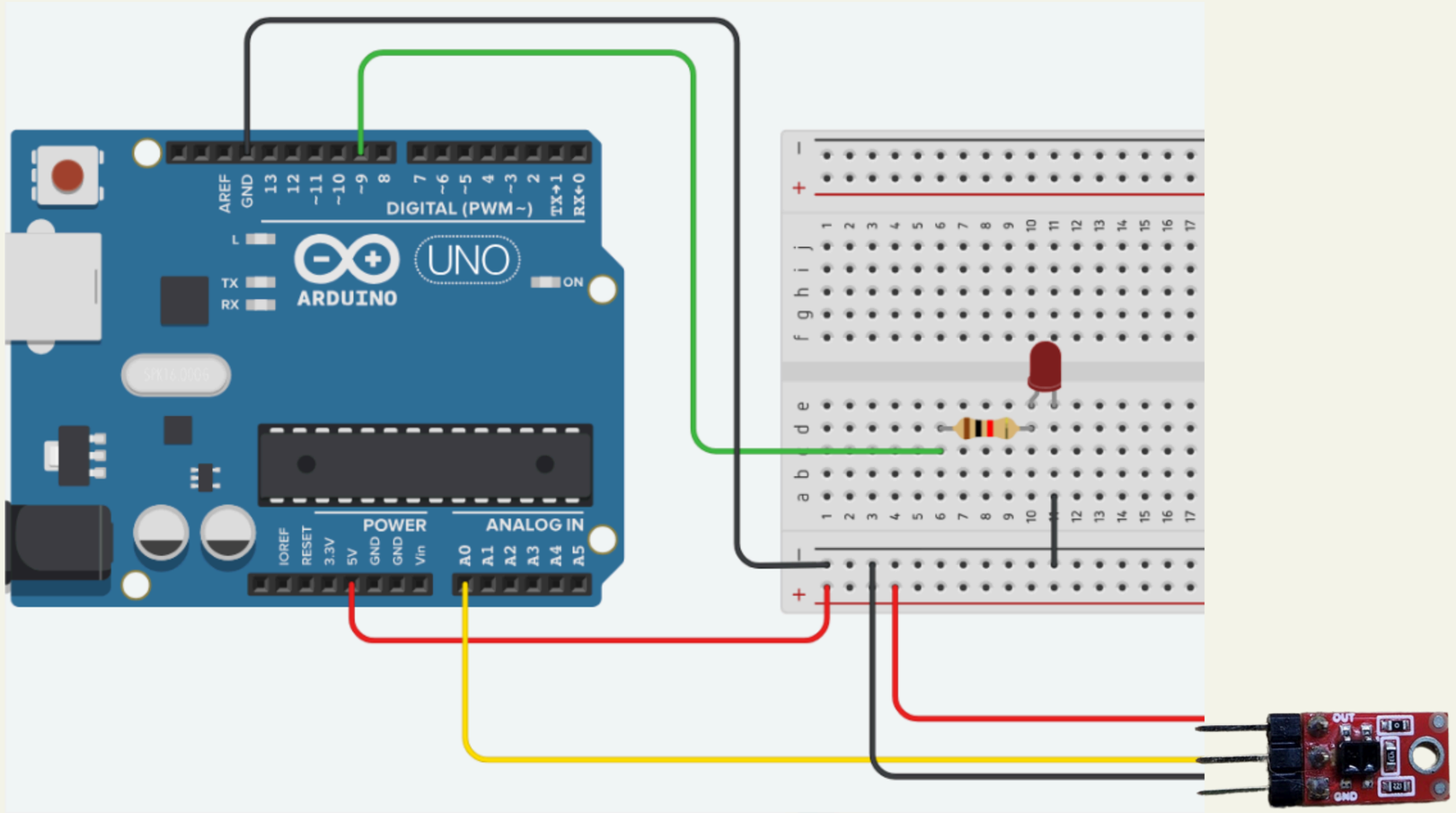
QTR-1 A/D



- **Serve como Sensor Óptico Reflexivo, porém tem melhor resposta e velocidade;**
- **O Sensor de Refletância também pode desenvolver um sinal Digital ou Analógico com a mesma função .**

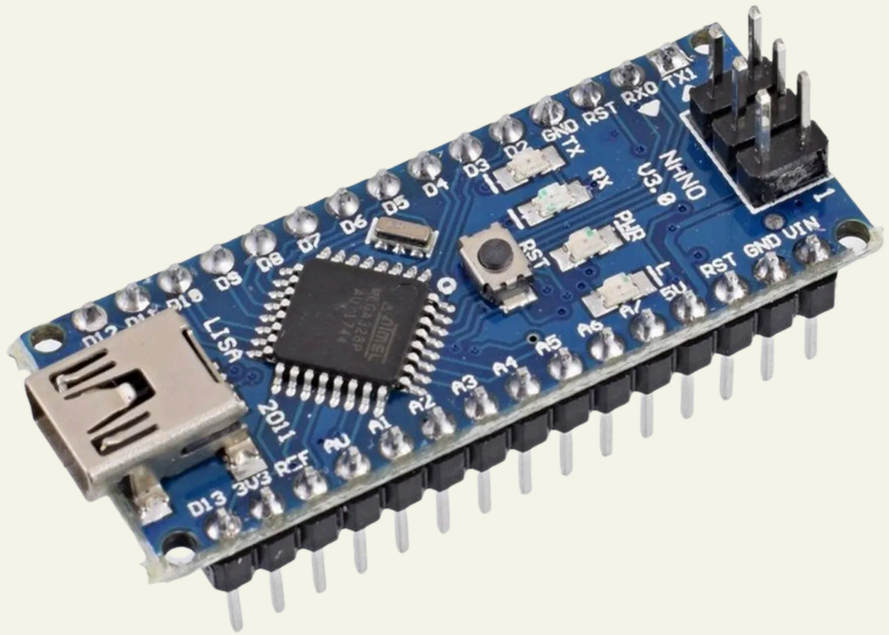
SENSOR ÓPTICO REFLEXIVO

Exemplo de Analógico



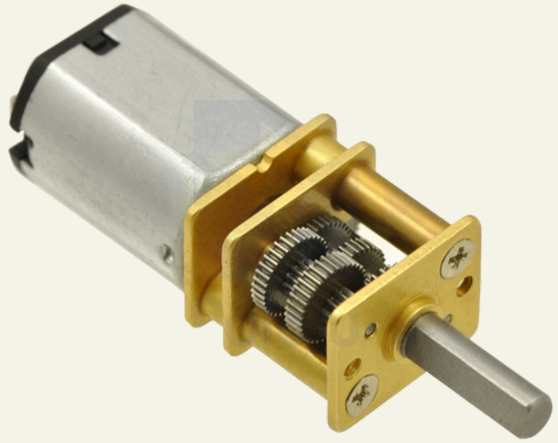
```
1   int  led 9; // Pino do led
2   int sensor = A0; // Ligado ao pino "coletor" do sensor optico
3   int leitura, valor;// Armazena informaçoes sobre a leitura do sensor
4
5   void setup() {
6       Serial.begin(9600);
7       pinMode(led, OUTPUT); // Define o pino do led como saída
8       pinMode(sensor, INPUT); // Define o pino do sensor como entrada
9   }
10  void loop(){
11    //Le as informações do pino do sensor
12    leitura = analogRead(sensor);
13    Serial.println(leitura);
14
15    // Verifica se o objeto foi detectado
16    if(leitura>800)
17    valor = 0;
18    else if(leitura<800 && leitura >= 600)
19    valor = 50;
20    else if (leitura<600 && leitura >= 400)
21    valor = 100;
22    else if (leitura<400 && leitura >= 200)
23    valor = 150;
24    else if (leitura < 250)
25    valor = 250;
26
27    analogWrite(led, valor);
28  }
```

ARDUINO NANO



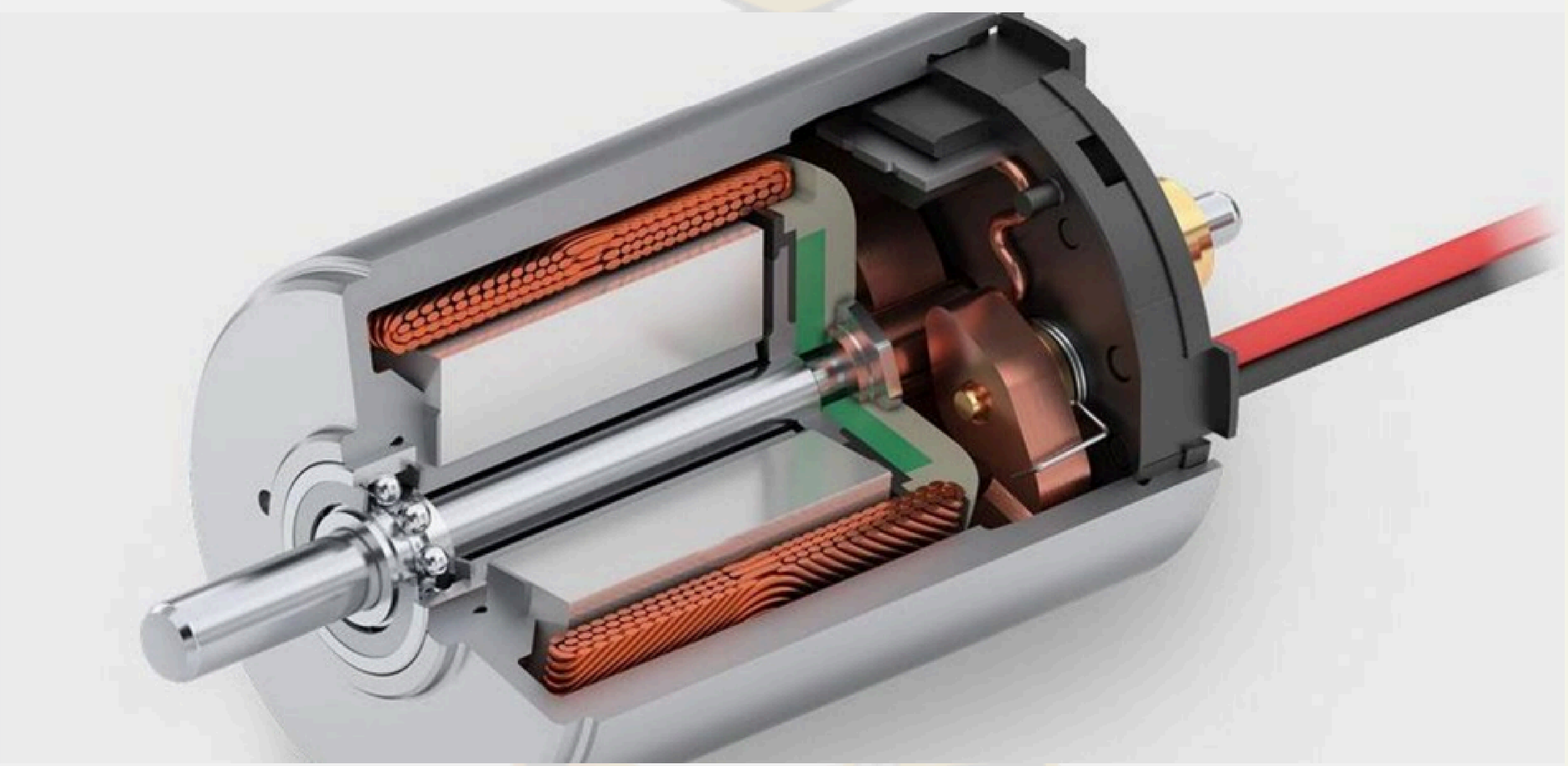
- **Tamanho reduzido;**
- **22 pinos de I/O (14 digitais, 8 analógicos);**
- **Conexão via USB Mini-B;**
- **Frequência de 16MHz.**

MOTORES DC (CORRENTE CONTÍNUA)

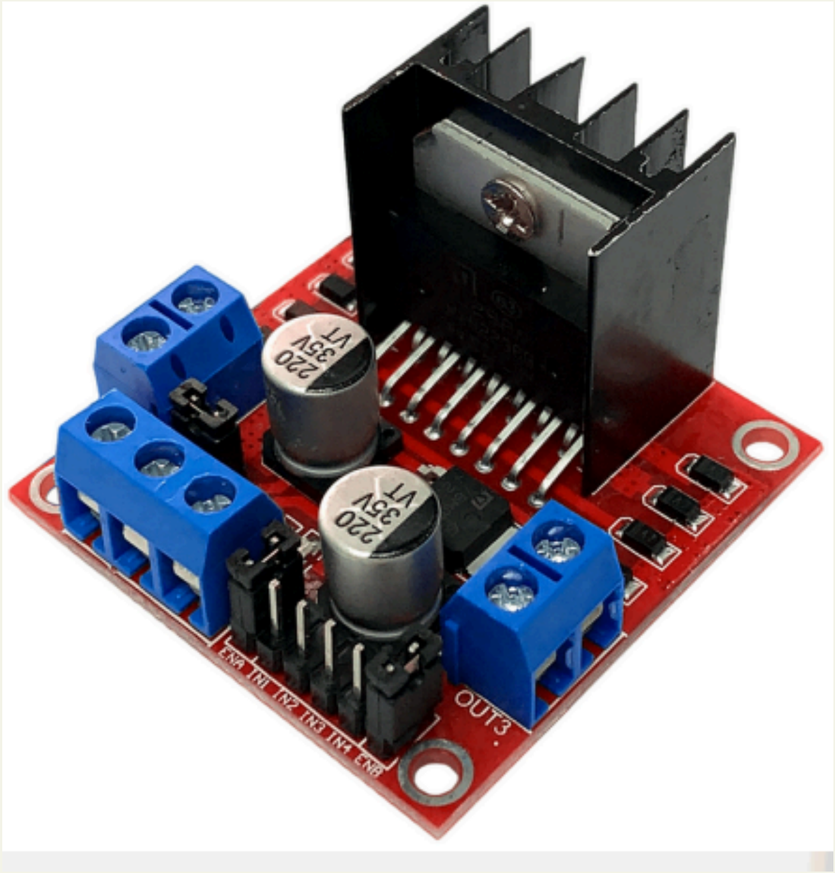


- **Conversão de energia: Elétrica → Mecânica rotacional.**
- **Mecanismo: Corrente nos enrolamentos do rotor gera campo magnético.**
- **Interação com campo do estator (ímãs permanentes/eletroímãs).**
- **Produz torque e rotação.**

MOTORES DC (CORRENTE CONTÍNUA)



Ponte H L298N



Função:

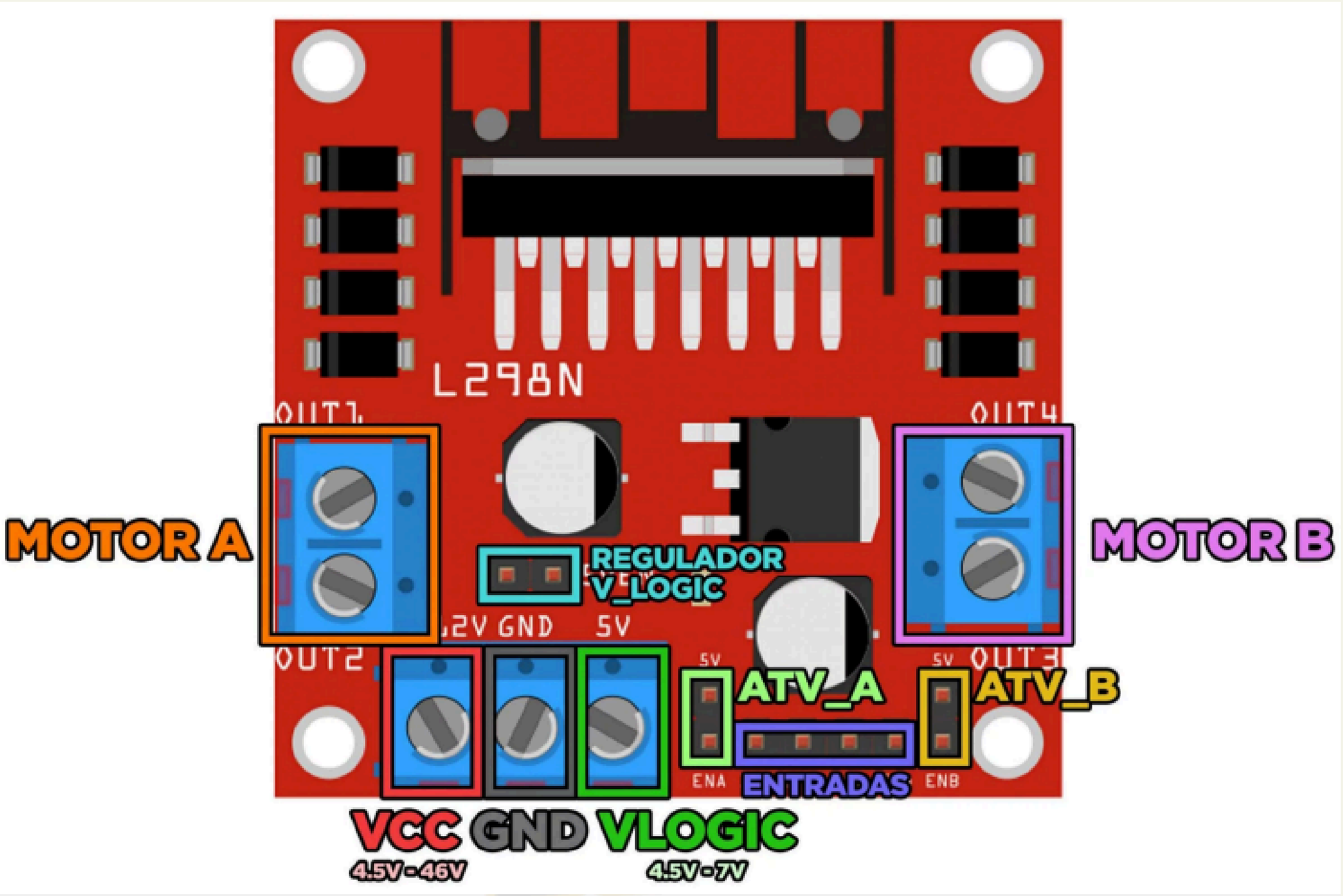
- CI de ponte H dupla para controle de motores

Aplicações:

- Motores DC (até 2)
- Motores de passo (1 bipolar)
- Solenoides

Especificações:

- Tensão: 5–46 V
- Corrente: 2 A contínuos / 3 A pico por canal


L298N
OUT1
OUT4
MOTOR A
MOTOR B
REGULADOR
V_LOGIC
12V GND 5V
OUT2
OUT3
5V
5V
ATV_A
ATV_B
ENA
ENTRADAS
ENB
VCC GND VLOGIC
4.5V-46V
4.5V-7V

Ponte H L298N

Pino	Função
VS	Alimentação dos motores (6-46V)
VCC	Lógica (5V)
GND	Terra
OUT1/2	Saídas Motor A
OUT3/4	Saídas Motor B
IN1-4	Controle de direção
ENA/ENB	Ativação PWM (velocidade)

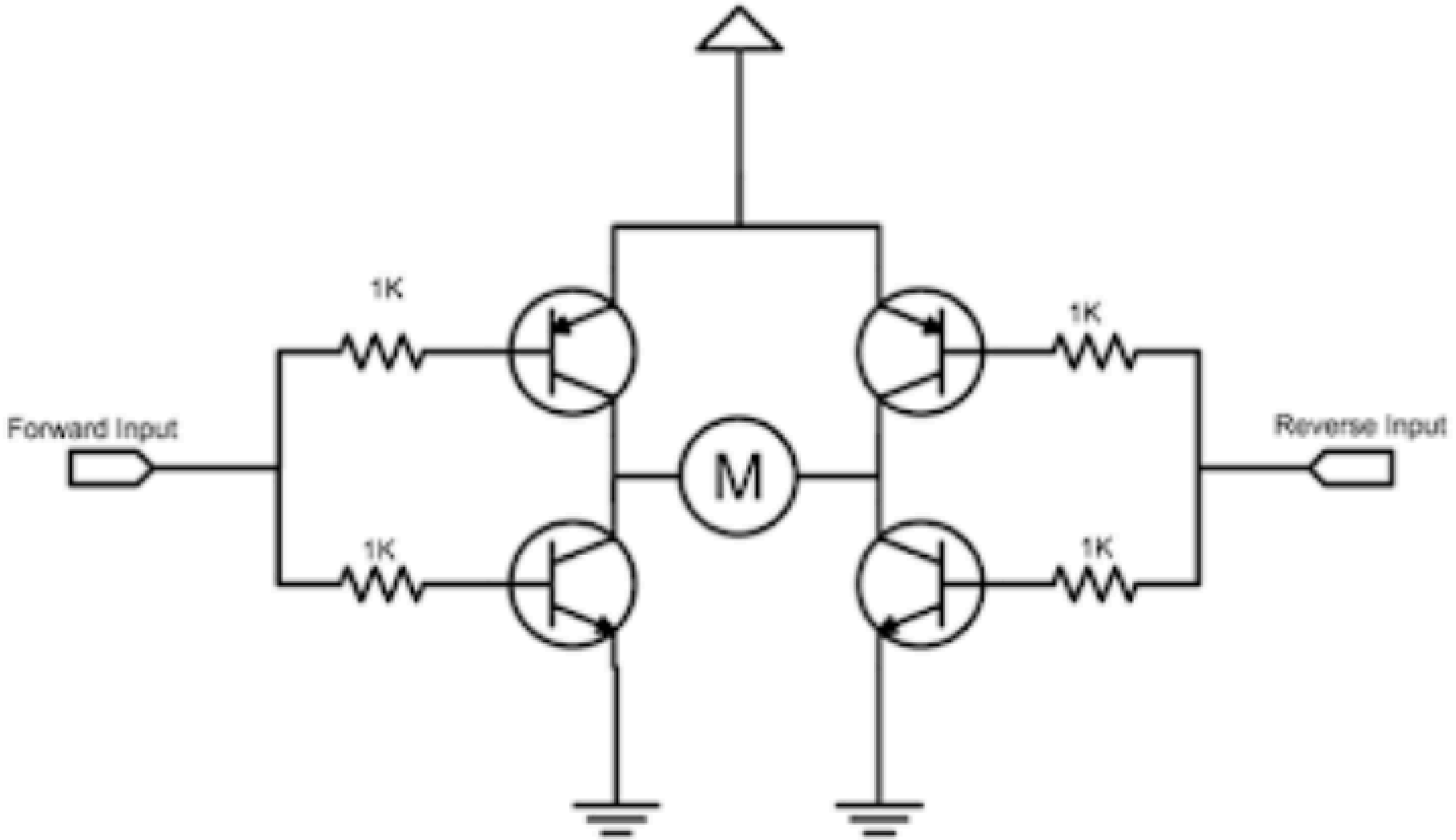
PONTE H L298N

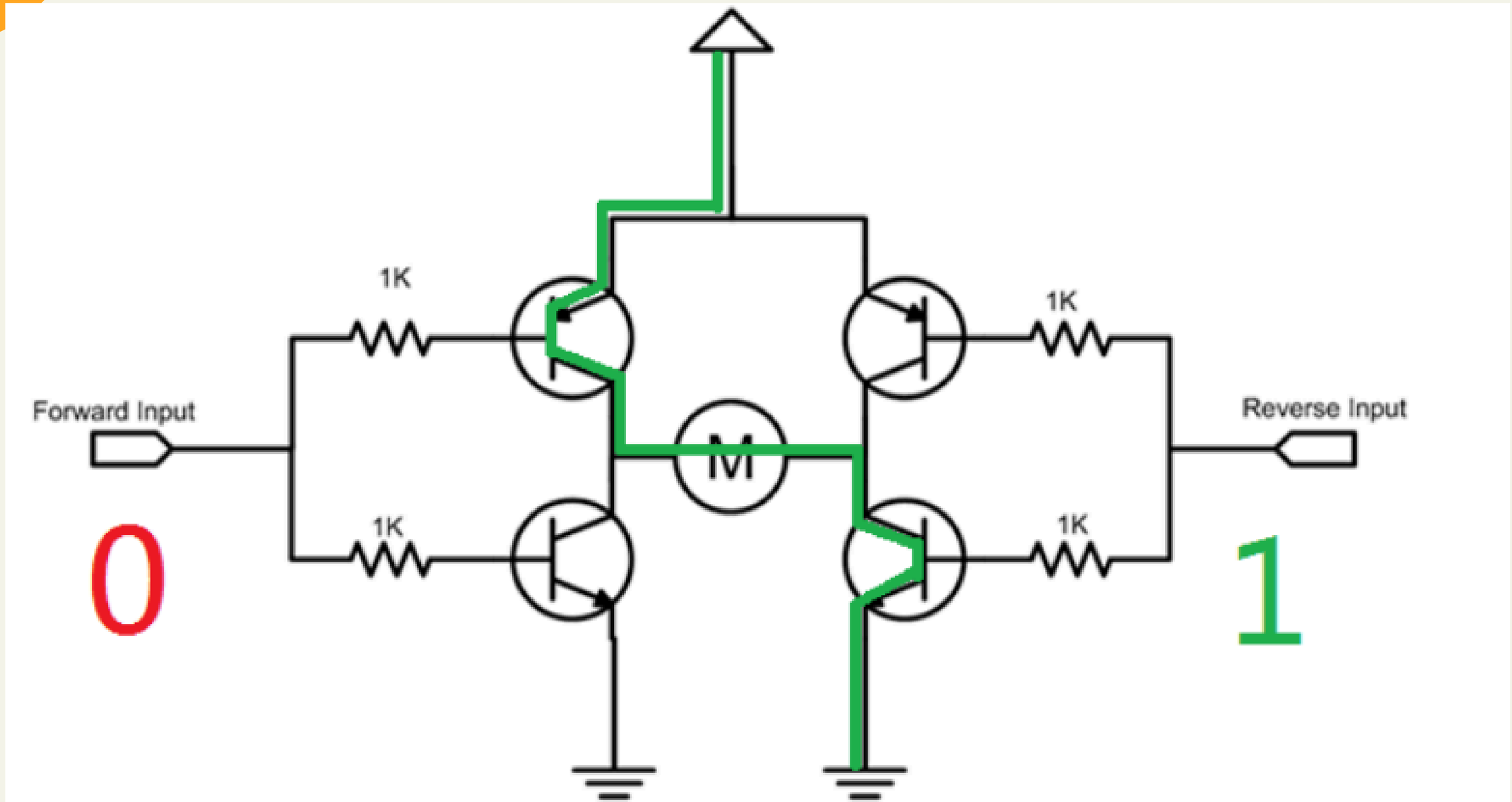
Funcionamento:

- 4 transistores controlam polaridade do motor:
- IN1=HIGH, IN2=LOW → Sentido horário
- IN1=LOW, IN2=HIGH → Sentido anti-horário

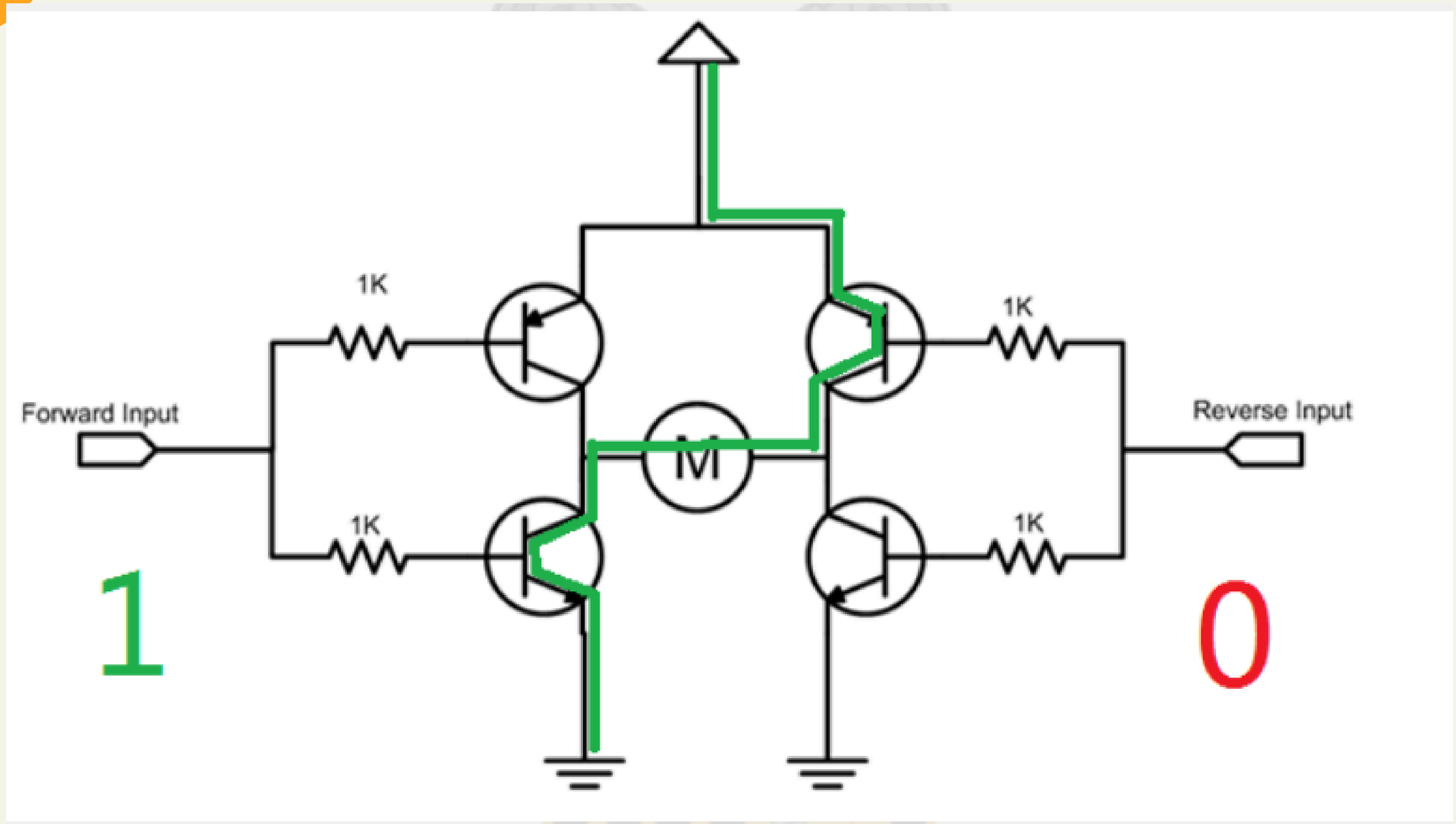
Controle de velocidade:

- PWM nos pinos ENA/ENB (0-255 no Arduino)

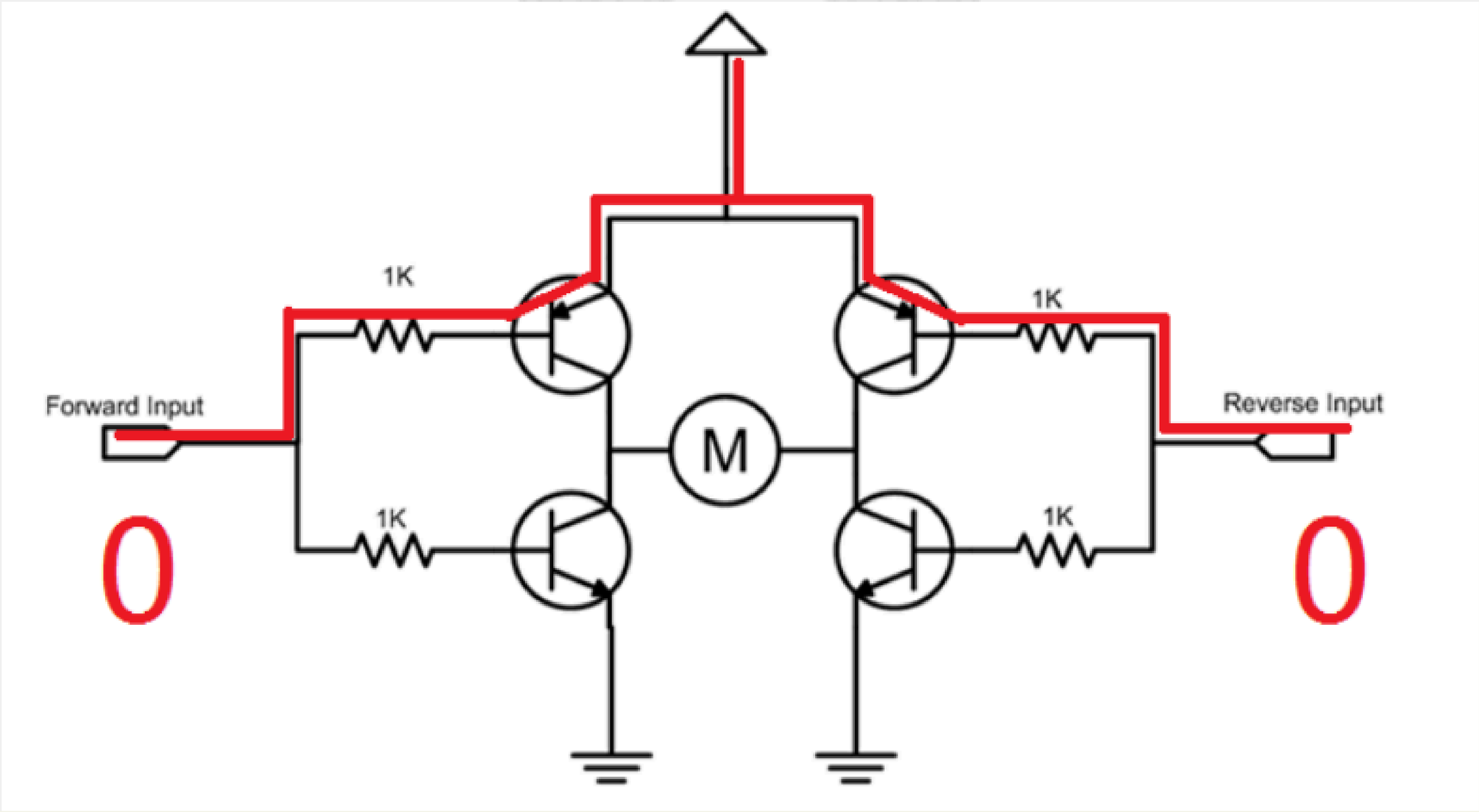

1K
1K
Forward Input
M
Reverse Input
1K
1K

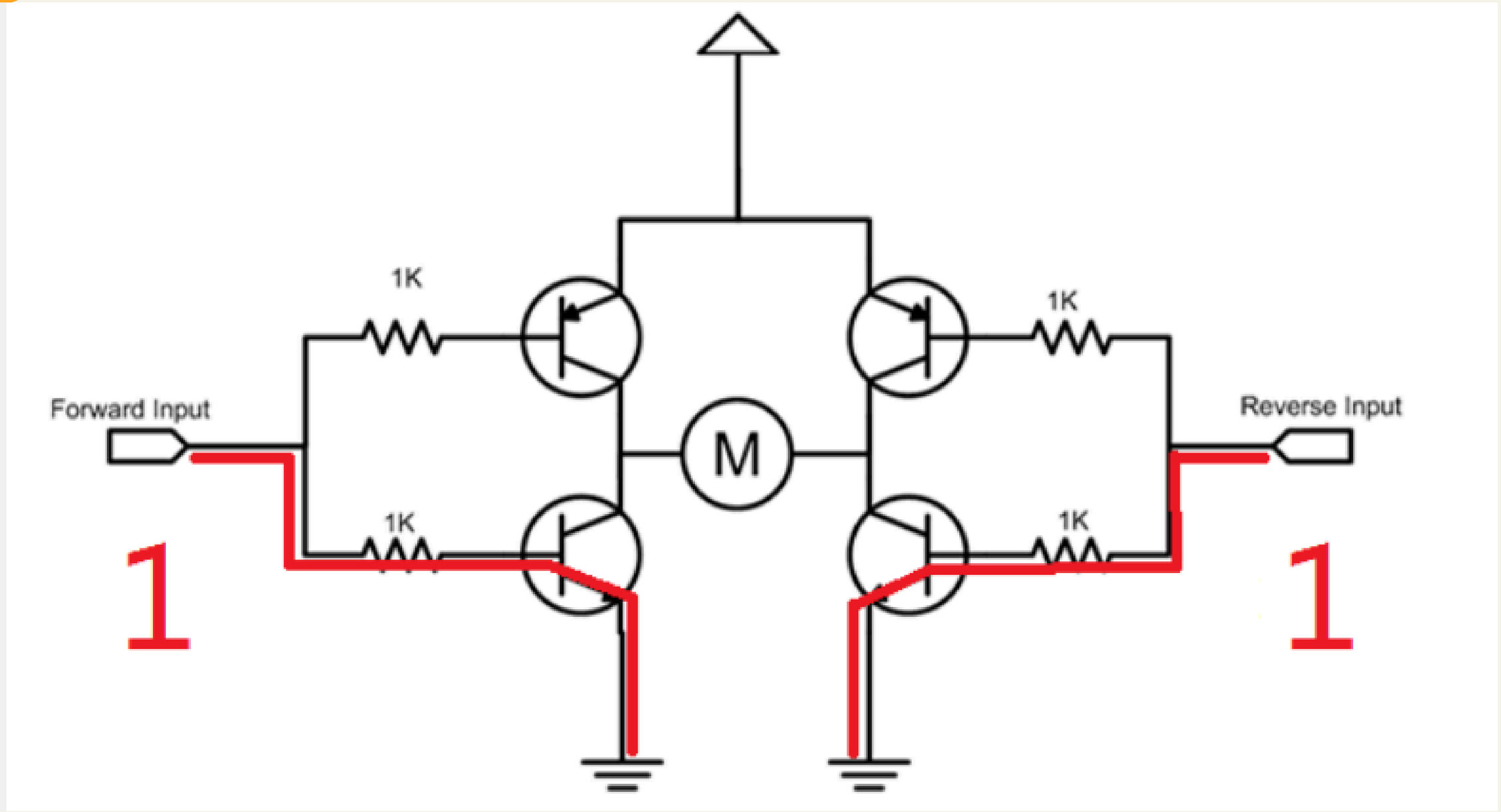


IN1=LOW, IN2=HIGH → Sentido anti-horário



IN1=HIGH, IN2=LOW → Sentido horário


1K
1K
Forward Input
Reverse Input
M
1K
1K
0
0


1K
1K
Forward Input
Reverse Input
M
1K
1K
1
1

```
#define IN1 9
#define IN2 10

void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
}

void loop() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
}
```

PWM

MODULAÇÃO POR LARGURA DE PULSO

Conceito Fundamental

- **O que é:**

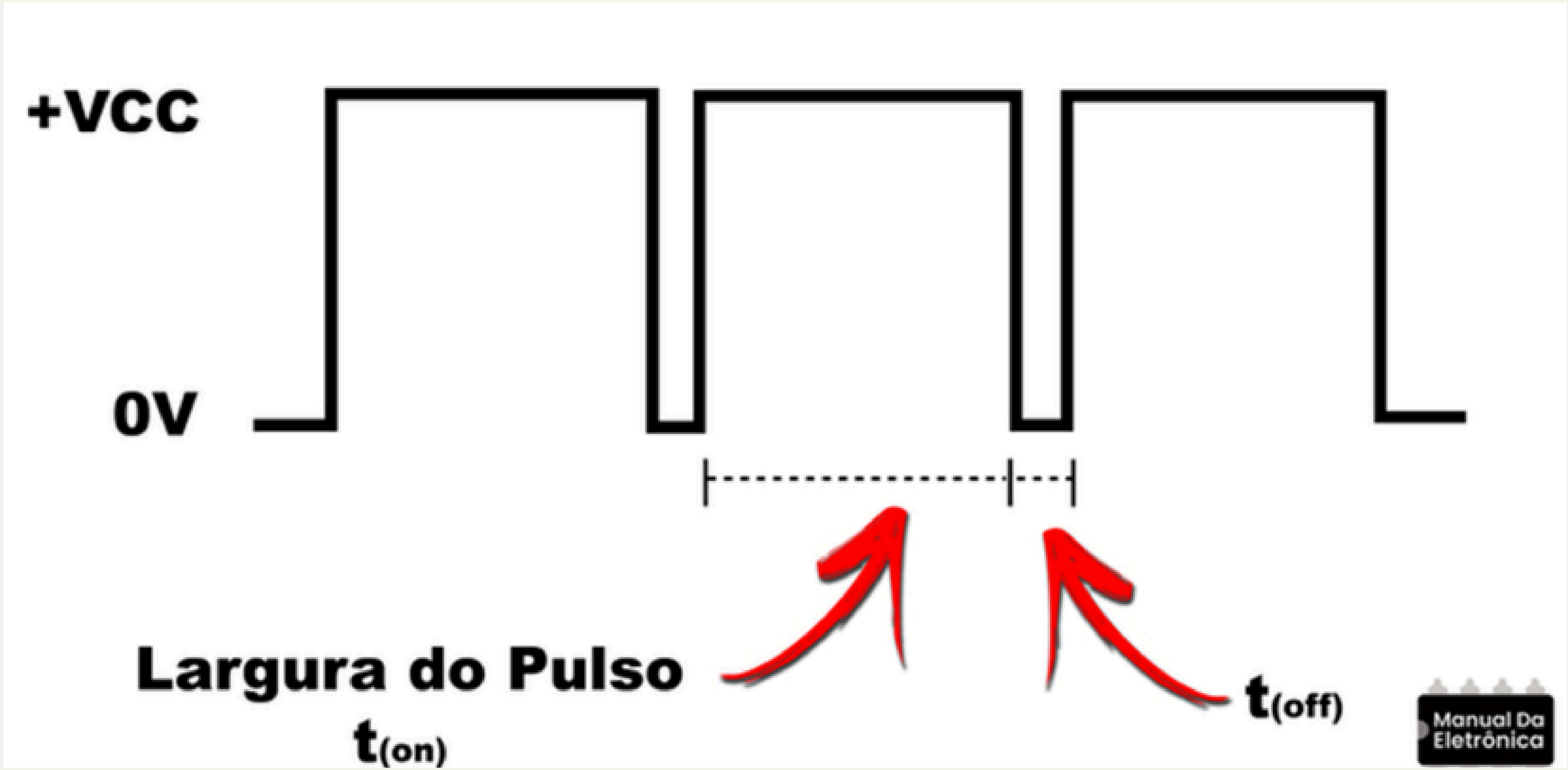
  Técnica para simular tensão analógica usando sinal digital.

- **Mecanismo:**

  Gera um sinal que alterna rapidamente entre HIGH (5V/3.3V) e LOW (0V).

- **Duty cycle:**

  Porcentagem do tempo em que o sinal fica em HIGH durante um ciclo.


+VCC
0V
Largura do Pulso
t(on)
t(off)
Manual Da Eletrônica

MODULAÇÃO POR LARGURA DE PULSO

Cálculo da Tensão Média

Exemplo (5V):

- Duty 25% → 5V × 0.25 = 1.25V
- Duty 50% → 5V × 0.50 = 2.5V
- Duty 75% → 5V × 0.75 = 3.75V > 1A

MODULAÇÃO POR LARGURA DE PULSO

Como Funciona na Prática

Controle de motor DC:

- Duty cycle 100% → motor a velocidade máxima.
- Duty cycle 30% → motor a ~30% da velocidade.

```
1  void setup() {
2    pinMode(9, OUTPUT);
3  }
4
5  void loop() {
6    analogWrite(9, 128);
7  }
```

```ino
// Pinos do Motor
const int IN1 = 7;      // Direção 1
const int IN2 = 8;      // Direção 2
const int ENA = 9;      // Controle PWM

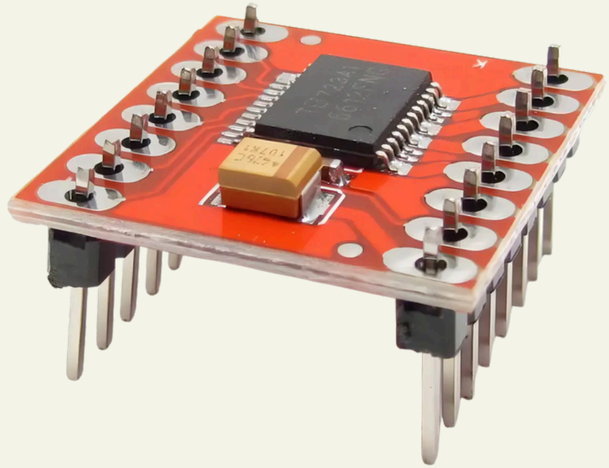
void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);

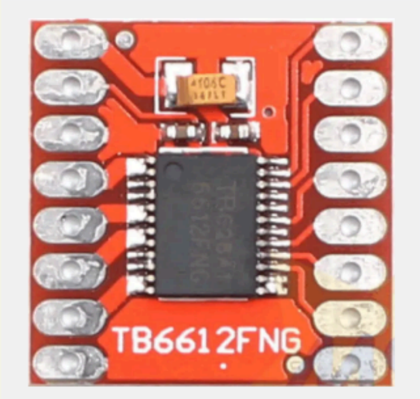
  // Configura direção fixa
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
}

void loop() {
  analogWrite(ENA, 64);   // 25% - RPM baixo

}
```

PONTE H TB6612FNG





Função:

- Driver dual para motores DC ou um motor de passo bipolar, baseado em MOSFETs;
- Alta eficiência (91–95%) e baixa queda de tensão (0.3V vs. 1.4V do L298N) .
- Corrente contínua: 1.2A por canal (pico: 3.2A);
- Tensão de operação: 2.5–13.5V para motores.

PONTE H TB6612FNG

Função STBY (Standby Global)

Propósito:

- Reduz consumo para <1 μA em modo standby (ideal para projetos com bateria);
- Protege contra operação acidental.

Uso Prático:

- Sempre ative no setup() do Arduino;
- Se omitido, os motores não funcionarão, mesmo com outros comandos.

```
1   void setup() {
2       pinMode(STBY, OUTPUT);
3       digitalWrite(STBY, HIGH);  // Ativa o driver
4   }
```

COMPARAÇÃO

Característica	TB6612FNG	L298N
Tecnologia	MOSFETs (baixa resistência)	Transistores bipolares (BJT)
Eficiência energética	90–95% (quase zero perda de tensão)	40–70% (queda de 1.4–2V)
Queda de tensão	0.3V (como resistor de 0.5Ω)	1.4–2V (perda significativa)
Corrente contínua	1.2A por canal (pico: 3.2A)	2A por canal (pico: 3A)
Tensão de operação	2.5–13.5V (motores) 2.7–5.5V (lógica)	4.5–46V (amplo alcance)
Dissipação de calor	Não requer dissipador	Exige dissipador grande
Modo standby	Sim (baixo consumo)	Não
Tamanho físico	Compacto (≈20mm²)	Grande (com dissipador)
Proteções integradas	Térmica e contra corrente reversa	Diodos flyback externos

Desafio

Sequência de movimentos:

1. Andar para frente
2. Girar 180° para esquerda
3. Andar para frente novamente
4. Girar 180° para direita
5. Parar completamente (Tem que voltar para o mesmo local que começou)

```ino
// Definição dos pinos - Motor A (ESQUERDA)
#define PWMA 5     // Controle PWM do motor A (velocidade)
#define AI1  9     // Controle de direção 1
#define AI2  4     // Controle de direção 2

// Definição dos pinos - Motor B (DIREITA)
#define PWMB 6     // Controle PWM do motor B (velocidade)
#define BI1  7     // Controle de direção 1
#define BI2  8     // Controle de direção 2

#define STBY 3     // Pino Standby (habilita/desabilita a ponte H)

void setup() {
  // Configura todos os pinos como saída
  pinMode(PWMA, OUTPUT);
  pinMode(AI1, OUTPUT);
  pinMode(AI2, OUTPUT);

  pinMode(PWMB, OUTPUT);
  pinMode(BI1, OUTPUT);
  pinMode(BI2, OUTPUT);

  pinMode(STBY, OUTPUT);

  // Habilita a ponte H (tira do modo standby)
  digitalWrite(STBY, HIGH);
}
```

```
29  void loop() {
30    // 1. MOVER PARA FRENTE (ambos motores no mesmo sentido)
31    // Motor esquerdo (A)
32    digitalWrite(AI1, HIGH);  // Define direção para frente
33    digitalWrite(AI2, LOW);   // O outro pino deve ficar LOW
34    analogWrite(PWMA, 150);   // Velocidade média (~60% da máxima)
35
36    // Motor direito (B)
37    digitalWrite(BI1, HIGH);  // Mesma direção do motor A
38    digitalWrite(BI2, LOW);
39    analogWrite(PWMB, 150);
40    delay(2000);              // Move por 2 segundos
41
42    // 2. GIRO 180° ESQUERDA (roda esquerda para trás, direita para frente)
43    // Motor esquerdo para trás
44    digitalWrite(AI1, LOW);   // Inverte a direção
45    digitalWrite(AI2, HIGH);
46    analogWrite(PWMA, 150);
47
48    // Motor direito continua para frente
49    digitalWrite(BI1, HIGH);
50    digitalWrite(BI2, LOW);
51    analogWrite(PWMB, 150);
52    delay(1000);              // Tempo de giro (ajustar conforme necessário)
53
```

```
54    // 3. MOVER PARA FRENTE NOVAMENTE
55    // Ambos motores para frente
56    digitalWrite(AI1, HIGH);
57    digitalWrite(AI2, LOW);
58    analogWrite(PWMA, 150);
59    digitalWrite(BI1, HIGH);
60    digitalWrite(BI2, LOW);
61    analogWrite(PWMB, 150);
62    delay(2000);
63
64    // 4. GIRO 180° DIREITA (roda direita para trás, esquerda para frente)
65    // Motor esquerdo para frente
66    digitalWrite(AI1, HIGH);
67    digitalWrite(AI2, LOW);
68    analogWrite(PWMA, 150);
69
70    // Motor direito para trás
71    digitalWrite(BI1, LOW);   // Inverte a direção
72    digitalWrite(BI2, HIGH);
73    analogWrite(PWMB, 150);
74    delay(1000);
75
76    // 5. PARADA COMPLETA
77    analogWrite(PWMA, 0);     // Desliga PWM (velocidade zero)
78    analogWrite(PWMB, 0);
79
80    while (1); // Loop infinito para parar a execução
81    // Remova esta linha se quiser que a sequência se repita
```